

`nullptr` 是指针字面值，2.3.2 节（第 47 页）将有更多关于指针和指针字面值的介绍。

2.1.3 节练习

练习 2.5：指出下述字面值的数据类型并说明每一组内几种字面值的区别：

- (a) `'a'`, `L'a'`, `"a"`, `L"a"`
- (b) `10`, `10u`, `10L`, `10uL`, `012`, `0xC`
- (c) `3.14`, `3.14f`, `3.14L`
- (d) `10`, `10u`, `10.`, `10e-2`

练习 2.6：下面两组定义是否有区别，如果有，请叙述之：

```
int month = 9, day = 7;
int month = 09, day = 07;
```

练习 2.7： 字面值表示何种含义？它们各自的数据类型是什么？

- (a) `"Who goes with F\145rgus?\012"`
- (b) `3.14e1L` (c) `1024f` (d) `3.14L`

练习 2.8：请利用转义序列编写一段程序，要求先输出 2M，然后转到新一行。修改程序使其先输出 2，然后输出制表符，再输出 M，最后转到新一行。

2.2 变量

变量提供一个具名的、可供程序操作的存储空间。C++ 中的每个变量都有其数据类型，数据类型决定着变量所占内存空间的大小和布局方式、该空间能存储的值的范围，以及变量能参与的运算。对 C++ 程序员来说，“变量 (variable)” 和 “对象 (object)” 一般可以互换使用。



2.2.1 变量定义

变量定义的基本形式是：首先是类型说明符 (type specifier)，随后紧跟由一个或多个变量名组成的列表，其中变量名以逗号分隔，最后以分号结束。列表中每个变量名的类型都由类型说明符指定，定义时还可以为一个或多个变量赋初值：

```
int sum = 0, value, // sum、value 和 units_sold 都是 int
    units_sold = 0; // sum 和 units_sold 初值为 0
Sales_item item;    // item 的类型是 Sales_item (参见 1.5.1 节，第 17 页)
// string 是一种库类型，表示一个可变长的字符序列
std::string book("0-201-78345-X"); // book 通过一个 string 字面值初始化
```

`book` 的定义用到了库类型 `std::string`，像 `iostream` (参见 1.2 节，第 6 页) 一样，`string` 也是在命名空间 `std` 中定义的，我们将在第 3 章中对 `string` 类型做更详细的介绍。眼下，只需了解 `string` 是一种表示可变长字符序列的数据类型就可以了。C++ 库提供了几种初始化 `string` 对象的方法，其中一种是把字面值拷贝给 `string` 对象 (参见 2.1.3 节，第 36 页)，因此在上例中，`book` 被初始化为 `0-201-78345-X`。

术语：何为对象？

C++程序员们在很多场合都会使用对象（object）这个名词。通常情况下，对象是指一块能存储数据并具有某种类型的内存空间。

一些人仅在与类有关的场景下才使用“对象”这个词。另一些人则已把命名的对象和未命名的对象区分开来，他们把命名了的对象叫做变量。还有一些人把对象和值区分开来，其中对象指能被程序修改的数据，而值（value）指只读的数据。

本书遵循大多数人的习惯用法，即认为对象是具有某种数据类型的内存空间。我们在使用对象这个词时，并不严格区分是类还是内置类型，也不区分是否命名或是否只读。

初始化

当对象在创建时获得了一个特定的值，我们说这个对象被初始化（initialized）了。用于初始化变量的值可以是任意复杂的表达式。当一次定义了两个或多个变量时，对象的名字随着定义也就马上可以使用了。因此在同一条定义语句中，可以用先定义的变量值去初始化后定义的其他变量。

```
//正确：price 先被定义并赋值，随后被用于初始化 discount
double price = 109.99, discount = price * 0.16;
//正确：调用函数 applyDiscount，然后用函数的返回值初始化 salePrice
double salePrice = applyDiscount(price, discount);
```

在 C++语言中，初始化是一个异常复杂的问题，我们也将反复讨论这个问题。很多程序员对于用等号=来初始化变量的方式倍感困惑，这种方式容易让人认为初始化是赋值的一种。事实上在 C++语言中，初始化和赋值是两个完全不同的操作。然而在很多编程语言中二者的区别几乎可以忽略不计，即使在 C++语言中有时这种区别也无关紧要，所以人们特别容易把二者混为一谈。需要强调的是，这个概念至关重要，我们也将后面不止一次提及这一点。

◀ 43



WARNING

初始化不是赋值，初始化的含义是创建变量时赋予其一个初始值，而赋值的含义是把对象的当前值擦除，而以一个新值来替代。

列表初始化

C++语言定义了初始化的好几种不同形式，这也是初始化问题复杂性的一个体现。例如，要想定义一个名为 units_sold 的 int 变量并初始化为 0，以下的 4 条语句都可以做到这一点：

```
int units_sold = 0;
int units_sold = {0};
int units_sold{0};
int units_sold(0);
```

作为 C++11 新标准的一部分，用花括号来初始化变量得到了全面应用，而在此之前，这种初始化的形式仅在某些受限的场合下才能使用。出于 3.3.1 节（第 88 页）将要介绍的原因，这种初始化的形式被称为列表初始化（list initialization）。现在，无论是初始化对象还是某些时候为对象赋新值，都可以使用这样一组由花括号括起来的初始值了。

C++
11

当用于内置类型的变量时，这种初始化形式有一个重要特点：如果我们使用列表初始化且初始值存在丢失信息的风险，则编译器将报错：

```
long double ld = 3.1415926536;
int a{ld}, b = {ld};           // 错误：转换未执行，因为存在丢失信息的危险
int c(ld), d = ld;             // 正确：转换执行，且确实丢失了部分值
```

使用 `long double` 的值初始化 `int` 变量时可能丢失数据，所以编译器拒绝了 `a` 和 `b` 的初始化请求。其中，至少 `ld` 的小数部分会丢失掉，而且 `int` 也可能存不下 `ld` 的整数部分。

刚刚所介绍的看起来无关紧要，毕竟我们不会故意用 `long double` 的值去初始化 `int` 变量。然而，像第 16 章介绍的一样，这种初始化有可能在不经意间发生。我们将在 3.2.1 节（第 76 页）和 3.3.1 节（第 88 页）对列表初始化做更多介绍。

默认初始化

如果定义变量时没有指定初值，则变量被默认初始化（default initialized），此时变量被赋予了“默认值”。默认值到底是什么由变量类型决定，同时定义变量的位置也会对此有影响。

44

如果是内置类型的变量未被显式初始化，它的值由定义的位置决定。定义于任何函数体之外的变量被初始化为 0。然而如 6.1.1 节（第 185 页）所示，一种例外情况是，定义在函数体内部的内置类型变量将不被初始化（uninitialized）。一个未被初始化的内置类型变量的值是未定义的（参见 2.1.2 节，第 33 页），如果试图拷贝或以其他方式访问此类值将引发错误。

每个类各自决定其初始化对象的方式。而且，是否允许不经初始化就定义对象也由类自己决定。如果类允许这种行为，它将决定对象的初始值到底是什么。

绝大多数类都支持无须显式初始化而定义对象，这样的类提供了一个合适的默认值。例如，以刚刚所见为例，`string` 类规定如果没有指定初值则生成一个空串：

```
std::string empty;           // empty 非显式地初始化为一个空串
Sales_item item;             // 被默认初始化的 Sales_item 对象
```

一些类要求每个对象都显式初始化，此时如果创建了一个该类的对象而未对其做明确的初始化操作，将引发错误。



定义于函数体内的内置类型的对象如果没有初始化，则其值未定义。类的对象如果没有显式地初始化，则其值由类确定。

2.2.1 节练习

练习 2.9：解释下列定义的含义。对于非法的定义，请说明错在何处并将其改正。

```
(a) std::cin >> int input_value; (b) int i = { 3.14 }
(c) double salary = wage = 9999. (d) int i = 3.14;
```

练习 2.10：下列变量的初值分别是什么？

```
std::string global_str;
int global_int;
int main()
{
    int local_int;
    std::string local_str;
}
```

提示：未初始化变量引发运行时故障

45

未初始化的变量含有一个不确定的值，使用未初始化变量的值是一种错误的编程行为并且很难调试。尽管大多数编译器都能对一部分使用未初始化变量的行为提出警告，但严格来说，编译器并未被要求检查此类错误。

使用未初始化的变量将带来无法预计的后果。有时我们足够幸运，一访问此类对象程序就崩溃并报错，此时只要找到崩溃的位置就很容易发现变量没被初始化的问题。另外一些时候，程序会一直执行完并产生错误的结果。更糟糕的情况是，程序结果时错时错、无法把握。而且，往无关的位置添加代码还会导致我们误以为程序对了，其实结果仍旧有错。



建议初始化每一个内置类型的变量。虽然并非必须这么做，但如果我们不能确保初始化后程序安全，那么这么做不失为一种简单可靠的方法。

2.2.2 变量声明和定义的关系



为了允许把程序拆分成多个逻辑部分来编写，C++语言支持分离式编译（separate compilation）机制，该机制允许将程序分割为若干个文件，每个文件可被独立编译。

如果将程序分为多个文件，则需要有在文件间共享代码的方法。例如，一个文件的代码可能需要使用另一个文件中定义的变量。一个实际的例子是 `std::cout` 和 `std::cin`，它们定义于标准库，却能被我们写的程序使用。

为了支持分离式编译，C++语言将声明和定义区分开来。声明（declaration）使得名字为程序所知，一个文件如果想使用别处定义的名字则必须包含对那个名字的声明。而定义（definition）负责创建与名字关联的实体。

变量声明规定了变量的类型和名字，在这一点上定义与之相同。但是除此之外，定义还申请存储空间，也可能会为变量赋一个初始值。

如果想声明一个变量而非定义它，就在变量名前添加关键字 `extern`，而且不要显式地初始化变量：

```
extern int i;    // 声明 i 而非定义 i
int j;          // 声明并定义 j
```

任何包含了显式初始化的声明即成为定义。我们能给由 `extern` 关键字标记的变量赋一个初始值，但是这么做也就抵消了 `extern` 的作用。`extern` 语句如果包含初始值就不再是声明，而变成定义了：

```
extern double pi = 3.1416; // 定义
```

在函数体内部，如果试图初始化一个由 `extern` 关键字标记的变量，将引发错误。

*Note*

变量能且只能被定义一次，但是可以被多次声明。

声明和定义的区别看起来也许微不足道，但实际上却非常重要。如果要在多个文件中使用同一个变量，就必须将声明和定义分离。此时，变量的定义必须出现在且只能出现在一个文件中，而其他用到该变量的文件必须对其进行声明，却绝对不能重复定义。

关于 C++语言对分离式编译的支持我们将在 2.6.3 节（第 67 页）和 6.1.3 节（第 186

46

页)中做更详细的介绍。

2.2.2 节练习

练习 2.11: 指出下面的语句是声明还是定义:

(a) `extern int ix = 1024;` 

(b) `int iy;` 

(c) `extern int iz;` 

关键概念: 静态类型

C++是一种静态类型 (statically typed) 语言, 其含义是在编译阶段检查类型。其中, 检查类型的过程称为类型检查 (type checking)。

我们已经知道, 对象的类型决定了对象所能参与的运算。在 C++语言中, 编译器负责检查数据类型是否支持要执行的运算, 如果试图执行类型不支持的运算, 编译器将报错并且不会生成可执行文件。

程序越复杂, 静态类型检查越有助于发现问题。然而, 前提是编译器必须知道每一个实体对象的类型, 这就要求我们在使用某个变量之前必须声明其类型。

2.2.3 标识符

C++的标识符 (identifier) 由字母、数字和下画线组成, 其中必须以字母或下画线开头。标识符的长度没有限制, 但是对大小写字母敏感:

```
// 定义 4 个不同的 int 变量
int somename, someName, SomeName, SOMENAME;
```

如表 2.3 和表 2.4 所示, C++语言保留了一些名字供语言本身使用, 这些名字不能被用作标识符。

同时, C++也为标准库保留了一些名字。用户自定义的标识符中不能连续出现两个下画线, 也不能以下画线紧连大写字母开头。此外, 定义在函数体外的标识符不能以下画线开头。

变量命名规范

变量命名有许多约定俗成的规范, 下面的这些规范能有效提高程序的可读性:

47

- 标识符要能体现实际含义。
- 变量名一般用小写字母, 如 `index`, 不要使用 `Index` 或 `INDEX`。
- 用户自定义的类名一般以大写字母开头, 如 `Sales_item`。
- 如果标识符由多个单词组成, 则单词间应有明显区分, 如 `student_loan` 或 `studentLoan`, 不要使用 `studentloan`。

Best Practices

对于命名规范来说, 若能坚持, 必将有效。

表 2.3: C++关键字				
alignas	continue	friend	register	true
alignof	decltype	goto	reinterpret_cast	try
asm	default	if	return	typedef
auto	delete	inline	short	typeid
bool	do	int	signed	typename
break	double	long	sizeof	union
case	dynamic_cast	mutable	static	unsigned
catch	else	namespace	static_assert	using
char	enum	new	static_cast	virtual
char16_t	explicit	noexcept	struct	void
char32_t	export	nullptr	switch	volatile
class	extern	operator	template	wchar_t
const	false	private	this	while
constexpr	float	protected	thread_local	
const_cast	for	public	throw	

表 2.4: C++操作符替代名					
and	bitand	compl	not_eq	or_eq	xor_eq
and_eq	bitor	not	or	xor	

2.2.3 节练习

练习 2.12: 请指出下面的名字中哪些是非法的?

(a) int double = 3.14;

(b) int _;

(c) int catch-22;

(d) int 1_or_2 = 1;

(e) double Double = 3.14;

2.2.4 名字的作用域



不论是在程序的什么位置，使用到的每个名字都会指向一个特定的实体：变量、函数、类型等。然而，同一个名字如果出现在程序的不同位置，也可能指向的是不同实体。

作用域 (scope) 是程序的一部分，在其中名字有其特定的含义。C++语言中大多数作用域都以花括号分隔。

同一个名字在不同的作用域中可能指向不同的实体。名字的有效区域始于名字的声明语句，以声明语句所在的作用域末端为结束。

一个典型的示例来自于 1.4.2 节（第 11 页）的程序：

```
#include <iostream>
int main()
{
    int sum = 0;
    // sum 用于存放从 1 到 10 所有数的和
    for (int val = 1; val <= 10; ++val)
        sum += val; // 等价于 sum = sum + val
```

```

std::cout << "Sum of 1 to 10 inclusive is "
          << sum << std::endl;
return 0;
}

```

这段程序定义了3个名字：main、sum 和 val，同时使用了命名空间名字 std，该空间提供了2个名字 cout 和 cin 供程序使用。

名字 main 定义于所有花括号之外，它和其他大多数定义在函数体之外的名字一样拥有全局作用域（global scope）。一旦声明之后，全局作用域内的名字在整个程序的范围内都可使用。名字 sum 定义于 main 函数所限定的作用域之内，从声明 sum 开始直到 main 函数结束为止都可以访问它，但是出了 main 函数所在的块就无法访问了，因此说变量 sum 拥有块作用域（block scope）。名字 val 定义于 for 语句内，在 for 语句之内可以访问 val，但是在 main 函数的其他部分就不能访问它了。

建议：当你第一次使用变量时再定义它

一般来说，在对象第一次被使用的地方附近定义它是一种好的选择，因为这样做有助于更容易地找到变量的定义。更重要的是，当变量的定义与它第一次被使用的地方很近时，我们也会赋给它一个比较合理的初始值。

嵌套的作用域

作用域能彼此包含，被包含（或者说被嵌套）的作用域称为内层作用域（inner scope），包含着别的作用域的作用域称为外层作用域（outer scope）。

作用域中一旦声明了某个名字，它所嵌套着的所有作用域中都能访问该名字。同时，允许在内层作用域中重新定义外层作用域已有的名字：

49

```

#include <iostream>
// 该程序仅用于说明：函数内部不宜定义与全局变量同名的新变量
int reused = 42; // reused 拥有全局作用域
int main()
{
    int unique = 0; // unique 拥有块作用域
    // 输出#1：使用全局变量 reused；输出 42 0
    std::cout << reused << " " << unique << std::endl;
    int reused = 0; // 新建局部变量 reused，覆盖了全局变量 reused
    // 输出#2：使用局部变量 reused；输出 0 0
    std::cout << reused << " " << unique << std::endl;
    // 输出#3：显式地访问全局变量 reused；输出 42 0
    std::cout << ::reused << " " << unique << std::endl;
    return 0;
}

```

输出#1 出现在局部变量 reused 定义之前，因此这条语句使用全局作用域中定义的名字 reused，输出 42 0。输出#2 发生在局部变量 reused 定义之后，此时局部变量 reused 正在作用域内（in scope），因此第二条输出语句使用的是局部变量 reused 而非全局变量，输出 0 0。输出#3 使用作用域操作符（参见 1.2 节，第 7 页）来覆盖默认的作用域规则，因为全局作用域本身并没有名字，所以当作用域操作符的左侧为空时，向全局作用域发出请求获取作用域操作符右侧名字对应的变量。结果是，第三条输出语句使用全局变量 reused，输出 42 0。



如果函数有可能用到某全局变量，则不宜再定义一个同名的局部变量。

2.2.4 节练习

练习 2.13: 下面程序中 j 的值是多少?

```
int i = 42;
int main()
{
    int i = 100;
    int j = i;
}
```

练习 2.14: 下面的程序合法吗? 如果合法, 它将输出什么?

```
int i = 100, sum = 0;
for (int i = 0; i != 10; ++i)
    sum += i;
std::cout << i " " << sum << std::endl;
```

2.3 复合类型

复合类型 (compound type) 是指基于其他类型定义的类型。C++ 语言有几种复合类型, 本章将介绍其中的两种: 引用和指针。

与我们已经掌握的变量声明相比, 定义复合类型的变量要复杂很多。2.2 节 (第 38 页) 提到, 一条简单的声明语句由一个数据类型和紧随其后的一个变量名列表组成。其实更通用的描述是, 一条声明语句由一个基本数据类型 (base type) 和紧随其后的一个声明符 (declarator) 列表组成。每个声明符命名了一个变量并指定该变量为与基本数据类型有关的某种类型。

目前为止, 我们所接触的声明语句中, 声明符其实就是变量名, 此时变量的类型也就是声明的基本数据类型。其实还可能有更复杂的声明符, 它基于基本数据类型得到更复杂的类型, 并把它指定给变量。

2.3.1 引用



C++11 中新增了一种引用: 所谓的“右值引用 (rvalue reference)”, 我们将在 13.6.1 节 (第 471 页) 做更详细的介绍。这种引用主要用于内置类。严格来说, 当我们使用术语“引用 (reference)”时, 指的其实是“左值引用 (lvalue reference)”。

引用 (reference) 为对象起了另外一个名字, 引用类型引用 (refers to) 另外一种类型。通过将声明符写成 &d 的形式来定义引用类型, 其中 d 是声明的变量名:

```
int ival = 1024;
int &refVal = ival;    // refVal 指向 ival (是 ival 的另一个名字)
int &refVal2;          // 报错: 引用必须被初始化
```



50

